Azure Deployment Guide (2026 Edition)

Step-by-Step CI/CD Setup for Django 5.2 on Azure App Service using GitHub Actions

[Django 5.2 LTS] [Python 3.11] [Azure App Service Linux] [PostgreSQL Flexible Server]

1 Phase 1: Unlock Azure Security Gate (SCM Basic Auth)

1. Log into the Azure Portal (https://portal.azure.com) and open your App Service.
2. In the left sidebar under 'Settings', click 'Configuration' (or 'Environment variables').
3. Click the 'General settings' tab at the top.
4. Scroll down to the 'Basic Auth' section.
5. Toggle 'SCM Basic Auth Publishing Credentials' to ON.
6. Click the 'Save' button at the top of the blade.

Note: In 2026, Basic Auth is off by default. Enabling SCM Basic Auth is mandatory for Publish Profiles.

2 Phase 2: Get Master Key (Publish Profile)

1. In Azure Portal, click 'Overview' on the left menu to return to App Service dashboard.
2. On the top toolbar, click 'Get publish profile' to download '<app-name>.PublishSettings'.
3. Open this downloaded file in any text editor (Notepad, VS Code).
4. Copy EVERY SINGLE line of XML text inside this file (Ctrl+A, Ctrl+C).

3 Phase 3: Hide Key in GitHub Secrets Vault

1. Open your browser to your GitHub repo: https://github.com/Driftingdane76/knowledgebase
2. Click the 'Settings' tab at the top of the repository.
3. In the left sidebar under 'Security', click 'Secrets and variables' -> 'Actions'.
4. Click the green 'New repository secret' button.
5. Under Name, type exactly: AZURE_WEBAPP_PUBLISH_PROFILE
6. Under Secret, paste the complete block of XML code copied from Phase 2.
7. Click 'Add secret'.

Phase 4: Mandatory Azure App Service Environment Settings

Configure under: App Service -> Settings -> Configuration / Environment variables

Environment Variable Name	Required Production Value / Purpose
SCM_DO_BUILD_DURING_DEPLOYMENT	true (CRITICAL: Triggers pip install -r requirements.txt)
DJANGO_SETTINGS_MODULE	core.settings
DEBUG	False
SECRET_KEY	<50+ random characters for cryptographic security>
DB_ENGINE	django.db.backends.postgresql
DB_NAME	<your-azure-postgresql-database-name>
DB_USER / DB_PASSWORD	<your-database-admin-username> / <password>
DB_HOST / DB_PORT	<your-server>.postgres.database.azure.com / 5432
ALLOWED_HOSTS	<your-app-name>.azurewebsites.net
REQUIRE_HTTPS	True (Enforces secure HTTPS cookies behind SSL)

! Mandatory Startup Command (General Settings -> Startup Command)

```
gunicorn --bind=0.0.0.0 --timeout 600 --workers 4 core.wsgi:application
```

5 Phase 5: Create .github/workflows/azure-deploy.yml

1. In your local repository root, create folder: .github/workflows/
2. Inside workflows, create file: azure-deploy.yml
3. Paste the production-ready YAML pipeline specification (detailed on Page 3).
4. Replace 'YOUR_AZURE_APP_NAME' with your actual Azure App Service resource name.

Production Workflow YAML & Deployment Execution

```
name: Build and deploy Python app to Azure Web App
on:
  push:
    branches: [ main ]
  workflow_dispatch:
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }
      - name: Upload artifact for deployment
        uses: actions/upload-artifact@v4
        with:
          name: python-app
          path: |
            .
            !.git/ !.venv/ !venv/ !antenv/ !__pycache__/
            !*.sqlite3 !.env !production_env !.agents/ !AGENTS.md
            !.cursorrules !test_output_images/ !test_htmls/
  deploy:
    runs-on: ubuntu-latest
    needs: build
    environment: { name: 'Production' }
    steps:
      - uses: actions/download-artifact@v4
        with: { name: python-app }
      - name: 'Deploy to Azure Web App'
        uses: azure/webapps-deploy@v3
        with:
          app-name: 'YOUR_AZURE_APP_NAME'  # Replace with actual App Service name
          slot-name: 'Production'
          publish-profile: ${{ secrets.AZURE_WEBAPP_PUBLISH_PROFILE }}
```

6 Phase 6: Commit, Deploy & Execute Database Migrations

1. In Command Prompt (cmd.exe), commit and push the pipeline:
   `git add .github/workflows/azure-deploy.yml && git commit -m "Add pipeline" && git push`
2. Monitor real-time logs under the 'Actions' tab on GitHub.
3. Once deployment turns GREEN, run database migrations in Azure Portal:
   `App Service -> Development Tools -> SSH -> type: python manage.py migrate`
4. Your application is now live at: https://<your-app-name>.azurewebsites.net